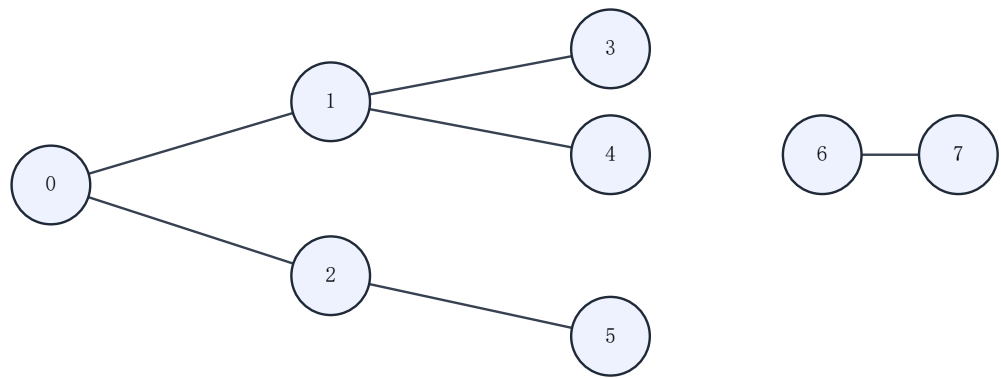


高级数据结构第二次作业

学号：202533354 姓名：ZHOU MINGYUAN

1. 图遍历与连通分量

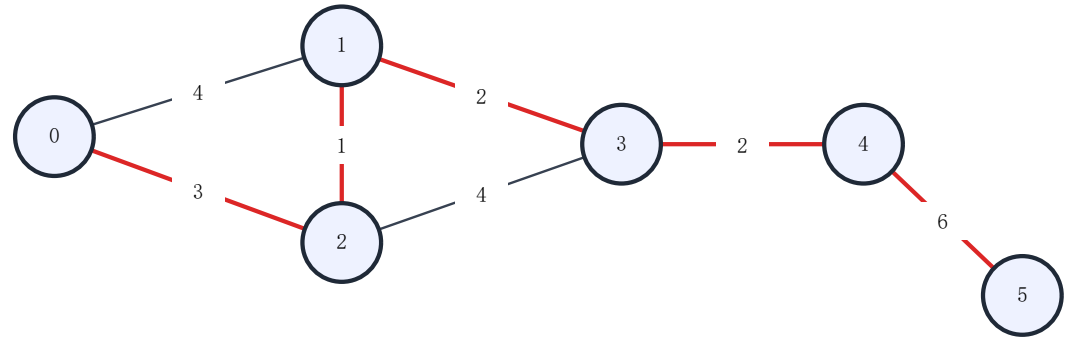
邻接点按编号从小到大访问。从顶点 0 开始：



- (a) DFS 顺序：0, 1, 3, 4, 2, 5。BFS 顺序：0, 1, 2, 3, 4, 5。
- (b) 所有连通分量为：{0, 1, 2, 3, 4, 5} 和 {6, 7}。
- (c) 如果 connected() 按顶点编号 0 到 7 扫描，并对未访问顶点启动 DFS，则每次 DFS 的起点为 0 和 6。

2. Kruskal 最小生成树

原带权无向图及最终 MST 中的边以红色标出：



顺序	考虑的边	权重	是否加入	原因
1	(1, 2)	1	加入	连接两个不同分量
2	(1, 3)	2	加入	连接两个不同分量
3	(3, 4)	2	加入	连接两个不同分量
4	(0, 2)	3	加入	连接顶点 0 所在分量
5	(0, 1)	4	不加入	0-2-1 已连通，会形成环
6	(2, 3)	4	不加入	2-1-3 已连通，会形成环
7	(4, 5)	6	加入	连接顶点 5

最终最小生成树边集合：{(1, 2), (1, 3), (3, 4), (0, 2), (4, 5)}，总权重为 1+2+2+3+6=14。Kruskal 需要判断是否形成环，因为最小生成树必须连通且无环；若加入会形成环的边，不仅破坏树结构，也不会减少连接所有顶点所需的总权重。

3. Dijkstra 最短路径

顶点对应: A=0, B=1, C=2, D=3, E=4, F=5。以 A 为源点。

步骤	选定顶点	已确定集合 S	d(A)	d(B)	d(C)	d(D)	d(E)	d(F)	主要松弛结果
初始	-	{}	0	inf	inf	inf	inf	inf	初始化
1	A	{A}	0	4	2	inf	inf	inf	A->B=4, A->C=2
2	C	{A, C}	0	4	2	10	12	inf	C->D=10, C->E=12
3	B	{A, C, B}	0	4	2	9	12	inf	B->D 将 10 更新为 9
4	D	{A, C, B, D}	0	4	2	9	11	15	D->E=11, D->F=15
5	E	{A, C, B, D, E}	0	4	2	9	11	13	E->F 将 15 更新为 13
6	F	{A, C, B, D, E, F}	0	4	2	9	11	13	结束

(a) 最短距离: A=0, B=4, C=2, D=9, E=11, F=13。

(b) 从 A 到 F 的最短路径为 A -> B -> D -> E -> F, 总长度为 4+5+2+2=13。

(c) Dijkstra 依赖贪心性质: 一旦某个当前距离最小的顶点被选定, 它的最短距离就不再改变。负权边可能在后续通过另一条路径把已确定顶点的距离变小, 因此 Dijkstra 不适用于含负权边的图。

(d) 若增加边 <4, 1> 且权重为 -4, 应使用 Bellman-Ford 算法。它能够处理负权边, 并且可以检测是否存在从源点可达的负权环。

4. 快速排序

数组为 [8, 3, 1, 7, 0, 10, 2], 每次选当前子数组第一个元素作为 pivot。

(a) 第一次 partition: pivot=8, 所有小于 8 的元素被移动到左侧, 10 在右侧, 结果为 [2, 3, 1, 7, 0, 8, 10]。

0	1	2	3	4	5	6
2	3	1	7	0	8	10
递归层次		处理的子数组				
第 1 层		[8, 3, 1, 7, 0, 10, 2]				
第 2 层		[2, 3, 1, 7, 0] 和 [10]				
第 3 层		[0, 1] 和 [7, 3]				
第 4 层		[0], [1], [3], [7]				

(c) 最终排序结果为 [0, 1, 2, 3, 7, 8, 10]。

5. 归并排序

原数组：[38, 27, 43, 3, 9, 82, 10]。

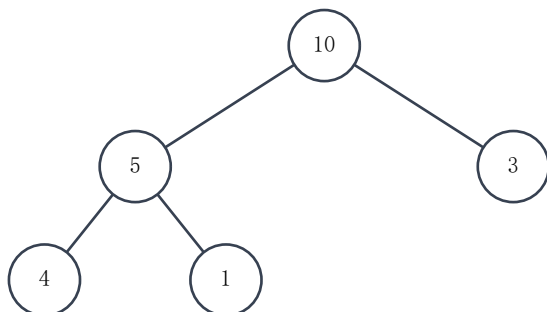
阶段	子数组
divide 1	[38, 27, 43, 3, 9, 82, 10] -> [38, 27, 43] 和 [3, 9, 82, 10]
divide 2	[38, 27, 43] -> [38] 和 [27, 43]; [3, 9, 82, 10] -> [3, 9] 和 [82, 10]
divide 3	[27, 43] -> [27] 和 [43]; [3, 9] -> [3] 和 [9]; [82, 10] -> [82] 和 [10]
合并步骤	合并结果
1	[27] + [43] -> [27, 43]
2	[38] + [27, 43] -> [27, 38, 43]
3	[3] + [9] -> [3, 9]
4	[82] + [10] -> [10, 82]
5	[3, 9] + [10, 82] -> [3, 9, 10, 82]
6	[27, 38, 43] + [3, 9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]

最终排序结果为 [3, 9, 10, 27, 38, 43, 82]。

6. 最大堆与堆排序

初始数组：[4, 10, 3, 5, 1]。自最后一个非叶子节点开始向前 heapify。

步骤	数组状态	说明
初始	[4, 10, 3, 5, 1]	原数组
heapify i=1	[4, 10, 3, 5, 1]	10 大于子节点 5 和 1，不变
heapify i=0	[10, 4, 3, 5, 1]	4 与较大孩子 10 交换
继续下滤	[10, 5, 3, 4, 1]	4 与较大孩子 5 交换，得到最大堆



排序步骤	数组状态	说明
1	[1, 5, 3, 4, 10] -> [5, 4, 3, 1, 10]	10 交换到末尾，对前 4 个元素重建堆
2	[1, 4, 3, 5, 10] -> [4, 1, 3, 5, 10]	5 进入有序区，对前 3 个元素重建堆
3	[3, 1, 4, 5, 10]	4 进入有序区，前 2 个元素已满足最大堆
4	[1, 3, 4, 5, 10]	3 进入有序区，排序完成

最终升序结果为 [1, 3, 4, 5, 10]。